

Abstract Interpretation of Computer Programs

Andre Quimper Osores

University of Michigan

aquimper@umich.edu

April 1, 2026

Overview

- 1 Motivation
- 2 Static Semantics of Programs
- 3 Abstract Interpretation of Programs
- 4 Consistent Abstract Interpretation
- 5 Iterative Solutions to the Fixpoint
- 6 Widening and Narrowing
- 7 Relational Domains
- 8 The Astrée Static Analyzer
- 9 Conclusion
- 10 Future Work

The Fundamental Challenge

- **Goal:** We want to determine if a program P satisfies a specific safety or correctness property Q (e.g., "no division by zero").

The Fundamental Challenge

- **Goal:** We want to determine if a program P satisfies a specific safety or correctness property Q (e.g., "no division by zero").
- **The Testing Problem:**
 - Testing covers only a finite subset of the execution space.
 - We cannot feasibly test the code with all possible concrete inputs for complex systems.

The Fundamental Challenge

- **Goal:** We want to determine if a program P satisfies a specific safety or correctness property Q (e.g., "no division by zero").
- **The Testing Problem:**
 - Testing covers only a finite subset of the execution space.
 - We cannot feasibly test the code with all possible concrete inputs for complex systems.
- **Theoretical limit:**
 - **Rice's Theorem:** All non-trivial semantic properties of programs are undecidable.
 - There is no perfect algorithm that works for all programs and always terminates with a "Yes/No" answer.

- We construct **approximations** of the program semantics.
- **Decidable:** The analysis must be guaranteed to terminate.
- **Sound:** Includes all possible behaviors (no false negatives).
- **Abstract Interpretation** provides the formal framework to ensure these mathematical guarantees.

To analyze a program, we define its actual behavior (the "concrete" domain):

- **Environment** (Env): A mapping from identifiers to values.

$$Env = \text{Identifiers} \rightarrow \text{Values}$$

To analyze a program, we define its actual behavior (the "concrete" domain):

- **Environment** (Env): A mapping from identifiers to values.

$$Env = \text{Identifiers} \rightarrow \text{Values}$$

- **Context Vector** (C_v): Maps each program location to the set of all possible environments occurring there.

$$C_v : L \rightarrow \mathcal{P}(Env)$$

To analyze a program, we define its actual behavior (the "concrete" domain):

- **Environment** (Env): A mapping from identifiers to values.

$$Env = \text{Identifiers} \rightarrow \text{Values}$$

- **Context Vector** (C_v): Maps each program location to the set of all possible environments occurring there.

$$C_v : L \rightarrow \mathcal{P}(Env)$$

- **Transfer Function** (F): Represents a single execution step. For a set of states S , $F(S)$ computes all states reachable in the next step.

To analyze a program, we define its actual behavior (the "concrete" domain):

- **Environment** (Env): A mapping from identifiers to values.

$$Env = \text{Identifiers} \rightarrow \text{Values}$$

- **Context Vector** (C_v): Maps each program location to the set of all possible environments occurring there.

$$C_v : L \rightarrow \mathcal{P}(Env)$$

- **Transfer Function** (F): Represents a single execution step. For a set of states S , $F(S)$ computes all states reachable in the next step.
- The total behavior of the program is the smallest set containing all reachable states, defined as the **Least Fixpoint** (lfp) of F :

$$C_v = \text{lfp } F = \bigcup_{n \in \mathbb{N}} F^n(\emptyset)$$

Problem: This is generally undecidable and incomputable.

An **Abstract Interpretation** $\mathcal{I}$ of a program P is defined by the 6-tuple:

$$\mathcal{I} = \langle \bar{C}, \sqsubseteq, \sqcup, \perp, \top, \text{Int} \rangle$$

- $\bar{C}$: The set of **Abstract Contexts**.

An **Abstract Interpretation** $\mathcal{I}$ of a program P is defined by the 6-tuple:

$$\mathcal{I} = \langle \bar{C}, \sqsubseteq, \sqcup, \perp, \top, \text{Int} \rangle$$

- $\bar{C}$: The set of **Abstract Contexts**.
- $(\bar{C}, \sqsubseteq)$: A **Complete Lattice** representing the partial ordering of "information" or "precision."

An **Abstract Interpretation** $\mathcal{I}$ of a program P is defined by the 6-tuple:

$$\mathcal{I} = \langle \bar{C}, \sqsubseteq, \sqcup, \perp, \top, \text{Int} \rangle$$

- $\bar{C}$: The set of **Abstract Contexts**.
- $(\bar{C}, \sqsubseteq)$: A **Complete Lattice** representing the partial ordering of "information" or "precision."
- $\sqcup$ (Join): The least upper bound operator, used to merge execution paths.

An **Abstract Interpretation** $\mathcal{I}$ of a program P is defined by the 6-tuple:

$$\mathcal{I} = \langle \bar{C}, \sqsubseteq, \sqcup, \perp, \top, \text{Int} \rangle$$

- $\bar{C}$: The set of **Abstract Contexts**.
- $(\bar{C}, \sqsubseteq)$: A **Complete Lattice** representing the partial ordering of "information" or "precision."
- $\sqcup$ (Join): The least upper bound operator, used to merge execution paths.
- $\perp, \top$: The infimum (no information) and supremum (anything could happen).

An **Abstract Interpretation** $\mathcal{I}$ of a program P is defined by the 6-tuple:

$$\mathcal{I} = \langle \bar{C}, \sqsubseteq, \sqcup, \perp, \top, \text{Int} \rangle$$

- $\bar{C}$: The set of **Abstract Contexts**.
- $(\bar{C}, \sqsubseteq)$: A **Complete Lattice** representing the partial ordering of "information" or "precision."
- $\sqcup$ (Join): The least upper bound operator, used to merge execution paths.
- $\perp, \top$: The infimum (no information) and supremum (anything could happen).

The Lattice Property

The ordering satisfies: $x \sqsubseteq y \iff x \sqcup y = y$. This ensures that as we "join" abstract states, we move up the lattice toward a sound over-approximation.

The function Int defines how basic instructions (on the program arcs) modify the abstract state.

- **Type Signature:** $\text{Int} : \text{Arcs} \times (\text{Arcs} \rightarrow \bar{C}) \rightarrow \bar{C}$
- It updates our abstract context vectors based on the program's control flow.
- **Monotonicity:** To guarantee convergence, Int must be order-preserving:

$$\forall x, y \in \bar{C} : x \sqsubseteq y \implies \text{Int}(x) \sqsubseteq \text{Int}(y)$$

The function Int defines how basic instructions (on the program arcs) modify the abstract state.

- **Type Signature:** $\text{Int} : \text{Arcs} \times (\text{Arcs} \rightarrow \bar{\mathcal{C}}) \rightarrow \bar{\mathcal{C}}$
- It updates our abstract context vectors based on the program's control flow.
- **Monotonicity:** To guarantee convergence, Int must be order-preserving:

$$\forall x, y \in \bar{\mathcal{C}} : x \sqsubseteq y \implies \text{Int}(x) \sqsubseteq \text{Int}(y)$$

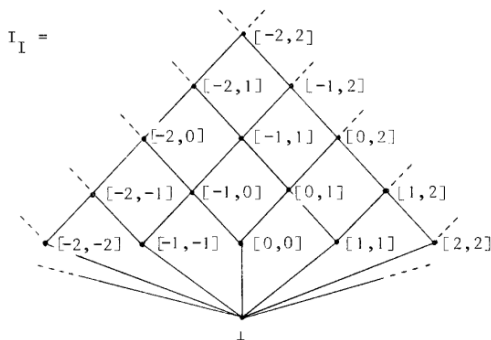
Approximation Guarantee

The semantics of the program are approximated by the abstract context vector $\bar{\mathcal{C}}_v$, which is the solution to the fixed-point equation:

$$\bar{\mathcal{C}}_v = \text{Int}(\bar{\mathcal{C}}_v)$$

A common abstract interpretation uses **Intervals** to approximate integer values.

- **Concrete:** $x \in \{1, 2, 5, 10\}$
- **Abstract:** $x \in [1, 10]$



The Join Operator ($\sqcup$):

$$[1, 3] \sqcup [5, 10] = [1, 10]$$

Note: We lose precision (the gap between 3 and 5), but we remain sound.

An abstract interpretation $\bar{\mathcal{I}}$ is **consistent** with a concrete interpretation $\mathcal{I}$ if it produces a context vector that correctly approximates the actual program behavior.

- We establish this consistency through a correspondence between the concrete domain (C) and the abstract domain (A).

An abstract interpretation $\bar{\mathcal{I}}$ is **consistent** with a concrete interpretation $\mathcal{I}$ if it produces a context vector that correctly approximates the actual program behavior.

- We establish this consistency through a correspondence between the concrete domain (C) and the abstract domain (A).
- This is formalized using two functions:

An abstract interpretation $\bar{\mathcal{I}}$ is **consistent** with a concrete interpretation $\mathcal{I}$ if it produces a context vector that correctly approximates the actual program behavior.

- We establish this consistency through a correspondence between the concrete domain (C) and the abstract domain (A).
- This is formalized using two functions:
 - **Abstraction (α)**: Maps a set of concrete states to the best possible abstract element.

An abstract interpretation $\bar{\mathcal{I}}$ is **consistent** with a concrete interpretation $\mathcal{I}$ if it produces a context vector that correctly approximates the actual program behavior.

- We establish this consistency through a correspondence between the concrete domain (C) and the abstract domain (A).
- This is formalized using two functions:
 - **Abstraction (α)**: Maps a set of concrete states to the best possible abstract element.
 - **Concretization (γ)**: Maps an abstract element back to the set of concrete states it represents.

An abstract interpretation $\bar{\mathcal{I}}$ is **consistent** with a concrete interpretation $\mathcal{I}$ if it produces a context vector that correctly approximates the actual program behavior.

- We establish this consistency through a correspondence between the concrete domain (C) and the abstract domain (A).
- This is formalized using two functions:
 - **Abstraction** (α): Maps a set of concrete states to the best possible abstract element.
 - **Concretization** (γ): Maps an abstract element back to the set of concrete states it represents.

The Soundness Goal

If our abstract result is $\bar{a}$, we want to guarantee that all actual execution states are contained within $\gamma(\bar{a})$.

The relationship between $\alpha : \mathcal{P}(\text{Env}) \rightarrow \bar{\mathcal{C}}$ and $\gamma : \bar{\mathcal{C}} \rightarrow \mathcal{P}(\text{Env})$ is defined by several key properties:

- **Monotonicity:** Both α and γ must be order-preserving:

$$S_1 \subseteq S_2 \implies \alpha(S_1) \sqsubseteq \alpha(S_2)$$

$$a_1 \sqsubseteq a_2 \implies \gamma(a_1) \subseteq \gamma(a_2)$$

The relationship between $\alpha : \mathcal{P}(Env) \rightarrow \bar{\mathcal{C}}$ and $\gamma : \bar{\mathcal{C}} \rightarrow \mathcal{P}(Env)$ is defined by several key properties:

- **Monotonicity:** Both α and γ must be order-preserving:

$$S_1 \subseteq S_2 \implies \alpha(S_1) \sqsubseteq \alpha(S_2)$$

$$a_1 \sqsubseteq a_2 \implies \gamma(a_1) \subseteq \gamma(a_2)$$

- **Extensivity (Soundness):** For any concrete set S , the abstraction followed by concretization must include the original set:

$$S \subseteq \gamma(\alpha(S))$$

The relationship between $\alpha : \mathcal{P}(Env) \rightarrow \bar{\mathcal{C}}$ and $\gamma : \bar{\mathcal{C}} \rightarrow \mathcal{P}(Env)$ is defined by several key properties:

- **Monotonicity:** Both α and γ must be order-preserving:

$$S_1 \subseteq S_2 \implies \alpha(S_1) \sqsubseteq \alpha(S_2)$$

$$a_1 \sqsubseteq a_2 \implies \gamma(a_1) \subseteq \gamma(a_2)$$

- **Extensivity (Soundness):** For any concrete set S , the abstraction followed by concretization must include the original set:

$$S \subseteq \gamma(\alpha(S))$$

- **Reductivity (Precision):** For any abstract element a , concretizing then abstracting should return to the same level of precision:

$$\alpha(\gamma(a)) \sqsubseteq a$$

The existence of a Galois connection ensures that our static analysis is a valid over-approximation of the program's runtime behavior.

Theorem (Soundness of Abstract Interpretation)

Let $\langle q, e \rangle$ be a state reached after n steps from an initial state i_s . If $\bar{C}_v(q)$ is the abstract context at location q , then:

$$\exists n \in \mathbb{N}, \exists i_s \in \text{InitialStates} \mid \langle q, e \rangle = n\text{-state}^n(i_s) \implies e \in \gamma(\bar{C}_v(q))$$

- **Interpretation:** If a variable can hold value v at line q in real life, v *must* be represented in the abstract state at q .

The existence of a Galois connection ensures that our static analysis is a valid over-approximation of the program's runtime behavior.

Theorem (Soundness of Abstract Interpretation)

Let $\langle q, e \rangle$ be a state reached after n steps from an initial state i_s . If $\bar{C}_v(q)$ is the abstract context at location q , then:

$$\exists n \in \mathbb{N}, \exists i_s \in \text{InitialStates} \mid \langle q, e \rangle = n\text{-state}^n(i_s) \implies e \in \gamma(\bar{C}_v(q))$$

- **Interpretation:** If a variable can hold value v at line q in real life, v *must* be represented in the abstract state at q .
- **Implication:** If our analysis says "no division by zero is possible in the abstract domain," it is **impossible** in the concrete domain.

If the abstract domain $\bar{C}$ is a complete lattice and the transfer function Int is monotonic, we can compute the solution iteratively.

- **The Bottom-Up Approach (AKS):** Starting from "no information" ($\perp$), we repeatedly apply the transition function.

$$X_0 = \perp, \quad X_{n+1} = \text{Int}(X_n)$$

If the abstract domain $\bar{C}$ is a complete lattice and the transfer function Int is monotonic, we can compute the solution iteratively.

- **The Bottom-Up Approach (AKS):** Starting from "no information" ($\perp$), we repeatedly apply the transition function.

$$X_0 = \perp, \quad X_{n+1} = \text{Int}(X_n)$$

- **Convergence:** For a finite lattice (or one satisfying the Ascending Chain Condition), there exists a finite k such that:

$$X_k = \text{Int}(X_k) = \text{lfp Int}$$

If the abstract domain $\bar{C}$ is a complete lattice and the transfer function Int is monotonic, we can compute the solution iteratively.

- **The Bottom-Up Approach (AKS):** Starting from "no information" ($\perp$), we repeatedly apply the transition function.

$$X_0 = \perp, \quad X_{n+1} = \text{Int}(X_n)$$

- **Convergence:** For a finite lattice (or one satisfying the Ascending Chain Condition), there exists a finite k such that:

$$X_k = \text{Int}(X_k) = \text{lfp Int}$$

Computational Significance

This turns a mathematical existence proof into an algorithm. We simply "run" the program in the abstract domain until the results stop changing.

- **The Problem:** Simple domains (like sign analysis) are finite, but many useful domains (like Intervals or Polyhedra) are **infinite** or have infinite height.

- **The Problem:** Simple domains (like sign analysis) are finite, but many useful domains (like Intervals or Polyhedra) are **infinite** or have infinite height.
- **Non-Termination:** On an infinite lattice, the Kleene sequence ($X_{n+1} = \text{Int}(X_n)$) might never reach a fixpoint.

- **The Problem:** Simple domains (like sign analysis) are finite, but many useful domains (like Intervals or Polyhedra) are **infinite** or have infinite height.
- **Non-Termination:** On an infinite lattice, the Kleene sequence ($X_{n+1} = \text{Int}(X_n)$) might never reach a fixpoint.
- While a specific finite lattice might work for a specific program, no single finite lattice can accurately analyze *all* programs.

- **The Problem:** Simple domains (like sign analysis) are finite, but many useful domains (like Intervals or Polyhedra) are **infinite** or have infinite height.
- **Non-Termination:** On an infinite lattice, the Kleene sequence ($X_{n+1} = \text{Int}(X_n)$) might never reach a fixpoint.
- While a specific finite lattice might work for a specific program, no single finite lattice can accurately analyze *all* programs.
- To gain powerful insights, we need infinite lattices.

- **The Problem:** Simple domains (like sign analysis) are finite, but many useful domains (like Intervals or Polyhedra) are **infinite** or have infinite height.
- **Non-Termination:** On an infinite lattice, the Kleene sequence ($X_{n+1} = \text{Int}(X_n)$) might never reach a fixpoint.
- While a specific finite lattice might work for a specific program, no single finite lattice can accurately analyze *all* programs.
- To gain powerful insights, we need infinite lattices.

The Solution

We use **Widening** to jump to an over-approximation quickly and **Narrowing** to refine that approximation.

Widening (∇)

A binary operator $\nabla : \bar{C} \times \bar{C} \rightarrow \bar{C}$ such that:

- **Covering:** $x \sqcup y \sqsubseteq x \nabla y$
- **Termination:** For any increasing chain, the sequence $W_0 = \perp, W_{n+1} = W_n \nabla \text{Int}(W_n)$ eventually stabilizes.

Narrowing ($\triangle$)

A binary operator $\triangle : \bar{C} \times \bar{C} \rightarrow \bar{C}$ used to recover precision:

- **Bounding:** $y \sqsubseteq x \implies y \sqsubseteq (x \triangle y) \sqsubseteq x$
- **Termination:** Ensures the decreasing sequence $N_0 = W_{\text{fix}}, N_{n+1} = N_n \triangle \text{Int}(N_n)$ stabilizes in finite steps.

The modern static analysis pipeline follows these steps:

- 1 **Abstraction:** Use Galois Connections (α, γ) to derive the abstract transfer functions from concrete semantics.

The modern static analysis pipeline follows these steps:

- 1 **Abstraction:** Use Galois Connections (α, γ) to derive the abstract transfer functions from concrete semantics.
- 2 **Convergence (Widening):** Compute an upward iteration with widening to find a **post-fixpoint** ($X \sqsupseteq \text{Int}(X)$) quickly.

$$W_{n+1} = W_n \nabla \text{Int}(W_n)$$

The modern static analysis pipeline follows these steps:

- 1 **Abstraction:** Use Galois Connections (α, γ) to derive the abstract transfer functions from concrete semantics.
- 2 **Convergence (Widening):** Compute an upward iteration with widening to find a **post-fixpoint** ($X \sqsupseteq \text{Int}(X)$) quickly.

$$W_{n+1} = W_n \nabla \text{Int}(W_n)$$

- 3 **Refinement (Narrowing):** Since widening often "over-shoots" and loses too much info, apply narrowing to move back down the lattice toward the fixpoint.

$$N_{n+1} = N_n \triangle \text{Int}(N_n)$$

The modern static analysis pipeline follows these steps:

- 1 **Abstraction:** Use Galois Connections (α, γ) to derive the abstract transfer functions from concrete semantics.
- 2 **Convergence (Widening):** Compute an upward iteration with widening to find a **post-fixpoint** ($X \sqsupseteq \text{Int}(X)$) quickly.

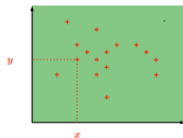
$$W_{n+1} = W_n \nabla \text{Int}(W_n)$$

- 3 **Refinement (Narrowing):** Since widening often "over-shoots" and loses too much info, apply narrowing to move back down the lattice toward the fixpoint.

$$N_{n+1} = N_n \triangle \text{Int}(N_n)$$

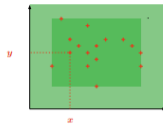
Example (Interval Widening)

$[0, 1] \nabla [0, 2] = [0, +\infty)$. We trade the specific bound for a guarantee that the loop analysis terminates.



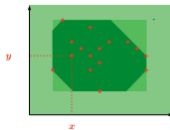
$$\begin{cases} x \geq 0 \\ y \geq 0 \end{cases}$$

(a) Signs Domain



$$\begin{cases} x \in [3, 27] \\ y \in [4, 32] \end{cases}$$

(b) Interval Domain



$$\begin{cases} 3 \leq x \leq 27 \\ x + y \leq 88 \\ 4 \leq y \leq 32 \\ x - y \leq 61 \end{cases}$$

(c) Octagon Domain

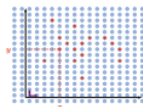


$$\begin{cases} 7x + 31y \leq 325 \\ 21x + 7y \geq 0 \end{cases}$$

(d) Polyhedra Domain

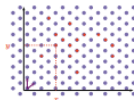
- **Non-Relational (a, b):** Fast and scalable, but cannot capture relationships between variables.
- **Relational (c, d):** More precise shapes, but requires more processing power to compute.

- **Core Concept:** Represents variables as $x \equiv a \pmod{b}$.
- **Utility:** Essential for analyzing memory alignment, loop unrolling, and stride patterns in arrays.



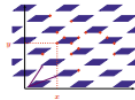
$$\begin{cases} x = 19 \pmod{88} \\ y = 19 \pmod{99} \end{cases}$$

(a) Simple Congruence



$$\begin{cases} 3x + 5y = 8 \pmod{27} \\ 2x - 9y = 23 \pmod{53} \end{cases}$$

(b) Relational



$$\begin{cases} 3x + 7y \in [2, 31] \pmod{10} \\ 2x - 5y \in [0, 9] \pmod{11} \end{cases}$$

(c) Trapezoidal

- **Simple:** Tracks $x = n \cdot \mathbb{Z} + a$ for individual variables.
- **Relational/Trapezoidal:** Captures modulo relationships between multiple variables (e.g., $x + y \equiv 0 \pmod{2}$).

The Cost of Precision

Choosing an abstract domain is an engineering trade-off between **Cost** and **False Alarms**.

Domain	Complexity	Precision	Scalability
Intervals	$O(n)$	Low	Very High
Octagons	$O(n^3)$	Medium	Medium/High
Polyhedra	$O(2^n)$	Very High	Low

- **Limitation:** No "universal" domain exists. High-precision domains often fail to terminate or exhaust memory on large projects, while low-precision domains produce too many "False Alarms" (spurious bug reports).

- **Beyond Intervals:** Standard intervals cannot capture relationships between variables (e.g., $x \leq y$).

- **Beyond Intervals:** Standard intervals cannot capture relationships between variables (e.g., $x \leq y$).
- **Linear Restraints (1978):** Cousot and Halbwachs introduced the **Polyhedra Domain**.

- **Beyond Intervals:** Standard intervals cannot capture relationships between variables (e.g., $x \leq y$).
- **Linear Restraints (1978):** Cousot and Halbwachs introduced the **Polyhedra Domain**.
 - Uses systems of linear inequalities: $\sum a_i x_i \leq b$.

- **Beyond Intervals:** Standard intervals cannot capture relationships between variables (e.g., $x \leq y$).
- **Linear Restraints (1978):** Cousot and Halbwachs introduced the **Polyhedra Domain**.
 - Uses systems of linear inequalities: $\sum a_i x_i \leq b$.
 - Capable of discovering complex invariants.

- **Beyond Intervals:** Standard intervals cannot capture relationships between variables (e.g., $x \leq y$).
- **Linear Restraints (1978):** Cousot and Halbwachs introduced the **Polyhedra Domain**.
 - Uses systems of linear inequalities: $\sum a_i x_i \leq b$.
 - Capable of discovering complex invariants.
 - **Limitation:** Computationally expensive (exponential time and space complexity).

- **Beyond Intervals:** Standard intervals cannot capture relationships between variables (e.g., $x \leq y$).
- **Linear Restraints (1978):** Cousot and Halbwachs introduced the **Polyhedra Domain**.
 - Uses systems of linear inequalities: $\sum a_i x_i \leq b$.
 - Capable of discovering complex invariants.
 - **Limitation:** Computationally expensive (exponential time and space complexity).
 - **Evaluation:** Evaluated on small but complex numerical algorithms (e.g., HEAPSORT) to prove loop invariants that intervals miss.

- **Beyond Intervals:** Standard intervals cannot capture relationships between variables (e.g., $x \leq y$).
- **Linear Restraints (1978):** Cousot and Halbwachs introduced the **Polyhedra Domain**.
 - Uses systems of linear inequalities: $\sum a_i x_i \leq b$.
 - Capable of discovering complex invariants.
 - **Limitation:** Computationally expensive (exponential time and space complexity).
 - **Evaluation:** Evaluated on small but complex numerical algorithms (e.g., HEAPSORT) to prove loop invariants that intervals miss.
 - **Limitations:**

- **Beyond Intervals:** Standard intervals cannot capture relationships between variables (e.g., $x \leq y$).
- **Linear Restraints (1978):** Cousot and Halbwachs introduced the **Polyhedra Domain**.
 - Uses systems of linear inequalities: $\sum a_i x_i \leq b$.
 - Capable of discovering complex invariants.
 - **Limitation:** Computationally expensive (exponential time and space complexity).
 - **Evaluation:** Evaluated on small but complex numerical algorithms (e.g., HEAPSORT) to prove loop invariants that intervals miss.
 - **Limitations:**
 - **Complexity:** Operations like the "Join" ($\sqcup$) require the Chernikova algorithm, which has exponential time/space complexity in the number of variables.

- **Beyond Intervals:** Standard intervals cannot capture relationships between variables (e.g., $x \leq y$).
- **Linear Restraints (1978):** Cousot and Halbwachs introduced the **Polyhedra Domain**.
 - Uses systems of linear inequalities: $\sum a_i x_i \leq b$.
 - Capable of discovering complex invariants.
 - **Limitation:** Computationally expensive (exponential time and space complexity).
 - **Evaluation:** Evaluated on small but complex numerical algorithms (e.g., HEAPSORT) to prove loop invariants that intervals miss.
 - **Limitations:**
 - **Complexity:** Operations like the "Join" ($\sqcup$) require the Chernikova algorithm, which has exponential time/space complexity in the number of variables.
 - **Scalability:** Generally limited to dozens of variables; unusable for entire industrial codebases without heavy partitioning.

- **The Octagon Domain (2006):** Antoine Miné introduced a "sweet spot" for efficiency in relational domains.

- **The Octagon Domain (2006):** Antoine Miné introduced a "sweet spot" for efficiency in relational domains.
 - Restricts constraints to $\pm x \pm y \leq c$.

- **The Octagon Domain (2006):** Antoine Miné introduced a "sweet spot" for efficiency in relational domains.
 - Restricts constraints to $\pm x \pm y \leq c$.
 - Forms a polyhedral shape with at most 8 sides in 2D (hence "octagon").

- **The Octagon Domain (2006):** Antoine Miné introduced a "sweet spot" for efficiency in relational domains.
 - Restricts constraints to $\pm x \pm y \leq c$.
 - Forms a polyhedral shape with at most 8 sides in 2D (hence "octagon").
 - Provides significant relational power while maintaining a cubic $O(n^3)$ cost.

- **The Octagon Domain (2006):** Antoine Miné introduced a "sweet spot" for efficiency in relational domains.
 - Restricts constraints to $\pm x \pm y \leq c$.
 - Forms a polyhedral shape with at most 8 sides in 2D (hence "octagon").
 - Provides significant relational power while maintaining a cubic $O(n^3)$ cost.
 - Strikes an excellent balance between precision and scalability for large programs.

- **The Octagon Domain (2006):** Antoine Miné introduced a "sweet spot" for efficiency in relational domains.
 - Restricts constraints to $\pm x \pm y \leq c$.
 - Forms a polyhedral shape with at most 8 sides in 2D (hence "octagon").
 - Provides significant relational power while maintaining a cubic $O(n^3)$ cost.
 - Strikes an excellent balance between precision and scalability for large programs.
- **Evaluation:** Successfully integrated into Astrée, making relational analysis feasible for large-scale code.

- **The Octagon Domain (2006):** Antoine Miné introduced a "sweet spot" for efficiency in relational domains.
 - Restricts constraints to $\pm x \pm y \leq c$.
 - Forms a polyhedral shape with at most 8 sides in 2D (hence "octagon").
 - Provides significant relational power while maintaining a cubic $O(n^3)$ cost.
 - Strikes an excellent balance between precision and scalability for large programs.
- **Evaluation:** Successfully integrated into Astrée, making relational analysis feasible for large-scale code.
- **Limitations:**

- **The Octagon Domain (2006):** Antoine Miné introduced a "sweet spot" for efficiency in relational domains.
 - Restricts constraints to $\pm x \pm y \leq c$.
 - Forms a polyhedral shape with at most 8 sides in 2D (hence "octagon").
 - Provides significant relational power while maintaining a cubic $O(n^3)$ cost.
 - Strikes an excellent balance between precision and scalability for large programs.
- **Evaluation:** Successfully integrated into Astrée, making relational analysis feasible for large-scale code.
- **Limitations:**
 - **Expressivity:** Cannot capture non-orthogonal relationships (e.g., $x + 2y \leq 5$).

- **The Octagon Domain (2006):** Antoine Miné introduced a "sweet spot" for efficiency in relational domains.
 - Restricts constraints to $\pm x \pm y \leq c$.
 - Forms a polyhedral shape with at most 8 sides in 2D (hence "octagon").
 - Provides significant relational power while maintaining a cubic $O(n^3)$ cost.
 - Strikes an excellent balance between precision and scalability for large programs.
- **Evaluation:** Successfully integrated into Astrée, making relational analysis feasible for large-scale code.
- **Limitations:**
 - **Expressivity:** Cannot capture non-orthogonal relationships (e.g., $x + 2y \leq 5$).
 - **Memory Overhead:** While better than Polyhedra, it still requires significantly more memory than non-relational intervals for programs with thousands of global variables.

The ultimate validation of these theories is their application to safety-critical embedded software.

- **Astrée (2003/2005):** A specialized static analyzer designed to prove the absence of Run-Time Errors (RTE) in C code.

The ultimate validation of these theories is their application to safety-critical embedded software.

- **Astrée (2003/2005):** A specialized static analyzer designed to prove the absence of Run-Time Errors (RTE) in C code.
- **Success Story:** Used to verify the primary flight control software of the **Airbus A340/A380** (hundreds of thousands of lines of C code).

The ultimate validation of these theories is their application to safety-critical embedded software.

- **Astrée (2003/2005):** A specialized static analyzer designed to prove the absence of Run-Time Errors (RTE) in C code.
- **Success Story:** Used to verify the primary flight control software of the **Airbus A340/A380** (hundreds of thousands of lines of C code).
- **Limitations:**

The ultimate validation of these theories is their application to safety-critical embedded software.

- **Astrée (2003/2005):** A specialized static analyzer designed to prove the absence of Run-Time Errors (RTE) in C code.
- **Success Story:** Used to verify the primary flight control software of the **Airbus A340/A380** (hundreds of thousands of lines of C code).
- **Limitations:**
 - **Domain Specificity:** Achieved "Zero False Alarms" only by adding specialized domains for digital filters and ellipsoids specific to control theory.

The ultimate validation of these theories is their application to safety-critical embedded software.

- **Astrée (2003/2005):** A specialized static analyzer designed to prove the absence of Run-Time Errors (RTE) in C code.
- **Success Story:** Used to verify the primary flight control software of the **Airbus A340/A380** (hundreds of thousands of lines of C code).
- **Limitations:**
 - **Domain Specificity:** Achieved "Zero False Alarms" only by adding specialized domains for digital filters and ellipsoids specific to control theory.
 - **Language Restrictions:** Does not support dynamic memory allocation (`malloc`) or recursion (standard in safety-critical embedded C, but limiting for general software).

- **The Core Problem:** Proving program properties is undecidable (Rice's Theorem), but safety-critical systems require guarantees.
- **Sound Over-approximation:** Abstract Interpretation provides a mathematical framework to ensure the analysis never misses a real bug.
- **Scaling to Infinity:** While Kleene iterations work for finite lattices, **Widening and Narrowing** allow us to analyze infinite domains (like Intervals or Polyhedra) efficiently.
- **Industrial Impact:** Tools like **Astrée** have successfully proven the absence of runtime errors in complex systems like the Airbus flight control software.

Future Research in Abstract Interpretation

Abstract interpretation continues to evolve to meet modern software challenges.

- **Solver Integration:** Combining abstract interpretation with **SAT/SMT solvers** to combine numerical abstraction with boolean logic and decision procedures.
- **Machine Learning Robustness:** Applying abstract interpretation to verify neural networks and ensure their safety against adversarial attacks.
- **Concurrency:** Modeling shared memory and thread interactions safely to avoid deadlocks and data races.

References I



P. Cousot and R. Cousot (1977).

Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints.

Proc. 4th ACM POPL, 238–252.



P. Cousot and N. Halbwachs (1978).

Automatic discovery of linear restraints among variables of a program.

Proc. 5th ACM POPL, 84–97.



P. Cousot and R. Cousot (1979).

Systematic design of program analysis frameworks.

Proc. 6th ACM POPL, 269–282.



P. Cousot and R. Cousot (1992).

Comparing the Galois connection and widening/narrowing approaches to abstract interpretation.

Proc. PLILP '92, LNCS 631, 269–295. Springer.



P. Cousot (1996).

Abstract interpretation: achievements and perspectives.

Proc. SSGRR, 1–23.



B. Blanchet et al. (2003).

A static analyzer for large safety-critical software.

Proc. ACM PLDI, 196–207.



P. Cousot et al. (2005).

The Astrée static analyzer.

Proc. ESOP 2005, LNCS 3444, 21–30. Springer.



A. Miné (2006).

The octagon abstract domain.

Higher-Order and Symbolic Computation, 19(1), 31–100.

Questions?

